

```

import fs from 'node:fs';
import path from 'node:path';
import crypto from 'node:crypto';
import { pathToFileURL } from 'node:url';
import { q, one } from '../db.js';
import { defineCapability, persistCapabilities } from './policy.js';
import { validateProvider, persistSlots } from './providers.js';
import { moduleFor } from './generated.js';

const loaded = new Map(); // key -> { manifest, module }

export function getPackage(key) { return loaded.get(key) ?? null; }
export function listPackages() { return [...loaded.values()].map(p => p.manifest)

// Read every package directory, validate its manifest, register its
// capabilities, and publish it so a business can install it.
export async function loadPackages(dir) {
  const entries = fs.readdirSync(dir, { withFileTypes: true }).filter(e => e.isDi

for (const entry of entries) {
  const base = path.join(dir, entry.name);
  const manifestPath = path.join(base, 'manifest.json');
  if (!fs.existsSync(manifestPath)) continue;

  const raw = fs.readFileSync(manifestPath, 'utf8');
  const manifest = JSON.parse(raw);
  validate(manifest);

  const entryPath = path.join(base, manifest.entry ?? 'index.js');
  const mod = fs.existsSync(entryPath)
    ? await import(pathToFileURL(entryPath).href)
    : {};

  if (manifest.kind === 'provider') validateProvider(manifest, mod);

  for (const cap of mod.capabilities ?? []) {
    defineCapability({ ...cap, packageKey: manifest.key });
  }

  loaded.set(manifest.key, { manifest, module: mod, dir: base });

  const hash = crypto.createHash('sha256').update(raw).digest('hex');
  await one(
    `insert into package (key, version, kind, name, summary, manifest, source,

```

```

    values ($1,$2,$3,$4,$5,$6,$7,$8)
    on conflict (key, version) do update set manifest = excluded.manifest,
      name = excluded.name, summary = excluded.summary, content_hash = exclude
    returning *`,
[manifest.key, manifest.version, manifest.kind, manifest.name,
  manifest.summary ?? null, JSON.stringify(manifest), 'local', hash]
);
}

await persistSlots();
await persistCapabilities();
return listPackages();
}

function validate(m) {
  const required = ['key', 'version', 'kind', 'name'];
  for (const f of required) if (!m[f]) throw new Error(`manifest missing ${f}`);
  if (!/^[a-z0-9][a-z0-9_-]*$/i.test(m.key)) throw new Error(`bad package key: ${m
  const kinds = ['app', 'plugin', 'connector', 'channel_app', 'provider', 'automa
  if (!kinds.includes(m.kind)) throw new Error(`unknown package kind: ${m.kind}`)
  if (m.kind === 'provider' && !m.fills) throw new Error(`${m.key} is a provider
  if (m.kind === 'channel_app') {
    if (!m.androidPackage) throw new Error(`${m.key} is a channel app with no and
    for (const [key, route] of Object.entries(m.routes ?? {})) {
      if (!route.write && !route.read) throw new Error(`${m.key} route "${key}" d
    }
    for (const key of m.carries ?? []) {
      if (!(m.routes ?? {})[key]) throw new Error(`${m.key} claims to carry "${ke
    }
  }
  for (const cap of m.capabilities ?? []) {
    if (!cap.startsWith(m.key + '.')) {
      throw new Error(`${m.key} declares capability outside its namespace: ${cap}
    }
  }
  for (const [name, def] of Object.entries(m.schema ?? {})) {
    if (!/^[a-z0-9_]+$/i.test(name)) throw new Error(`bad table name: ${name}`);
    if (!def.fields) throw new Error(`table ${name} declares no fields`);
    if ('business_id' in def.fields) throw new Error(`table ${name} may not decla
  }
}

export async function unloadAll() { loaded.clear(); }

// A manifest produced at runtime – by the builder, or by the importer – becomes
// a real package with real capabilities, through exactly the same validator.
export function registerGenerated(manifest) {

```

```
validate(manifest);
const mod = manifest.schema ? moduleFor(manifest) : { capabilities: [], rendere
for (const cap of mod.capabilities) defineCapability({ ...cap, packageKey: mani
loaded.set(manifest.key, { manifest, module: mod, dir: null });
return mod;
}
```